

GAINE - Tema 5 - Bases de datos de grafos (Neo4j)

1. Introduccion a Neo4j y Cypher

1.1. Arquitectura de Neo4j

EXAM=(2025J2.11)

El rendimiento de Neo4j es superior al de las bases de datos relacionales en escenarios donde las relaciones entre las entidades son elementos de primer nivel, tan críticos como los propios registros.

1.1.2. Terminología y conceptos Un **nodo** sería el equivalente a una fila en SQL.

EXAM=(2023S0.15)

Una **propiedad** sería el equivalente a una columna en SQL.

Una **relación** es una conexión entre dos nodos del grafo. Sería el equivalente a un join en SQL.

Características principales de Neo4j

1.2. Introducción a Cypher

1.2.3. Codificación de consultas sencillas EXAM=(2025J2.8,2025J2.10,2025J2.12)

La cláusula **MATCH** sirve para seleccionar elementos del grafo.

EXAM=(2023J2.9)

La sentencia **RETURN** devuelve un resultado. Si hay varias variables devueltas, las muestra en formato tabla.

Un **camino** (en inglés, *path*) en Neo4j/Cypher es una secuencia conectada de nodos y relaciones. Puede asignarse a una variable en un **MATCH** para devolverlo o manipularlo completo.

Dado este ejemplo:

```
CREATE
(ana:Persona {nombre: "Ana"}),
(bea:Persona {nombre: "Bea"}),
(carlos:Persona {nombre: "Carlos"}),
(ana)-[:SIGUE]->(bea),
(bea)-[:SIGUE]->(carlos);
```

La siguiente sentencia:

```
MATCH camino = (:Persona {nombre: "Ana"})-[:SIGUE]->(:Persona)-[:SIGUE]->(:Persona)
RETURN camino;
```

Devolverá:

Ana --SIGUE--> Bea --SIGUE--> Carlos

2. Importación de datos desde ficheros CSV en Neo4j

2.1

2.1.2. Marcado de claves principales unívocas

EXAM=(2023J2.11)

El NOT EXISTS evalúa que no se da una situación concreta.

Modelado en Neo4j

Operaciones CRUD en Neo4j

Operadores en Neo4j

Creación de nodos y relaciones en Neo4j

Búsqueda de información en Neo4j

Actualización de propiedades en Neo4j

3. Borrado de información en Neo4j

3.1

3.1.4

3.1.4.1. Cypher para especialización de relaciones

EXAM=(2022S0.15,2022S0.20)

COUNT(*) se usa para...

Si en un RETURN se incluye COUNT(*), Neo4j automáticamente agrega los resultados repetidos y muestra en su lugar el número de veces que se repite.

RETURN a.name, d.name, count(*)

// Resultado: (Harrison Ford, Indiana Jones, 5)

4. Optimización y rendimiento en Neo4j

4.1. Operadores

4.1.4. Operadores sobre cadenas de texto El operador `CONTAINS` comprueba si una cadena contiene una subcadena.

```
MATCH (p:Persona)
WHERE p.nombre CONTAINS "an"
RETURN p.nombre
```

El operador `ENDS WITH`...

`EXAM=(2024J2.11,2024S0.7)`

El operador `STARTS WITH` evalúa que el comienzo de una cadena de texto comienza por la cadena indicada tras el operador.

El operador `toUpper()`...

El operador `toLowerCase()`...

4.1.5. Operadores sobre listas `=EXAM(2022J2.20)`

El operador `IN` se usa para comprobar si un valor se encuentra dentro de una lista.

```
MATCH (p:Persona)
WHERE p.edad IN [20, 25, 30]
RETURN p
```

`=EXAM(2025S0.8)`

El operador `UNWIND` separa los elementos incluidos en una lista.

El operador `isEmpty()`...

`=EXAM(2022S0.16, 2022S0.17, 2022S0.18)`

El operador `size()` devuelve el número de elementos de una lista, patrón o camino.

Ejemplo:

```
MATCH (p:Persona)
RETURN p.nombre, size((p)-[:AMIGO_DE]->()) AS numAmigos
```

4.1.6. Operadores agregados `=EXAM(2023J2.8,2024J2.9,2024S0.8)`

`collect()` crea una lista a partir de una agregación de resultados. Las listas se identifican porque se imprimen con corchetes.

```
MATCH (a:Persona)-[:ACTUA]->(m:Pelicula)
RETURN a.nombre, collect(m.titulo) AS peliculas
```

=EXAM(2024J2.8)

DISTINCT elimina valores repetidos, de manera que solo queden valores únicos.

SUM()...

4.2

4.2.1. Definición de un nuevo nodo y una nueva relación

4.2.2. Uso de la función de creación de nodos y relaciones

=EXAM(EX.2025J2.9)

CREATE sirve para crear un nodo o una relación. En caso de que ya exista, lo duplica.

Ejemplo de creación de un par de nodos con una relación:

```
CREATE (u:Usuario)-[:SIGUE]->(v:Usuario);
```

Ejemplo de cómo establecer una relación entre todos los nodos existentes:

```
MATCH (u:Usuario), (v:Usuario) CREATE (u)-[:SIGUE]->(v);
```

Ejemplo de creación de una relación entre dos nodos existentes:

```
MATCH (u:Usuario {nombre: "Alice"}), (v:Usuario {nombre: "Bob"})  
CREATE (u)-[:SIGUE]->(v);
```

EXAM=(2023S0.14,2024S0.6)

MERGE sirve para crear una relación si no existe o mantenerla sin duplicarla.

```
MERGE (c:Cliente {idCliente: 'Bertoldo'})  
RETURN c;
```

4.3

4.3.2. Anidamiento de búsquedas EXAM=(2023J2.10)

Si en una relación se añade asterisco al final del nombre, la relación se puede dar en tantos saltos como haga falta.

EXAM=(2023S0.13)

Cuando hay un número tras el asterisco, se limita el número de saltos. Se puede definir un rango empleando la sintaxis `[:RELACION*a..b]` (donde a y b son números positivos, y $a < b$).

```
MATCH (u:Usuario {nombre: 'Carlos'})-[:SIGUE*1..2]->(d:Director)-[:DIRIGIO]->(p:Pelicula)
RETURN DISTINCT p.titulo AS Recomendaciones
```

4.3.3. Ordenación y limitación

```
EXAM=(2023S0.12)
```

ORDER BY sirve para ordenar. Si va seguido por ASC se ordena de menor a mayor, y si va seguido por DESC se ordena de mayor a menor.

```
EXAM=(2023S0.12, 2024J2.10)
```

La cláusula LIMIT establece el máximo de resultados a mostrar en una consulta.

4.3.4

4.3.5. Subconsultas EXAM=(2025S0.10)

En las subconsultas la cláusula WITH a secas se usa para que una variable pueda ser empleada en el ámbito de dos subconsultas.

```
MATCH (u:Usuario {nombre: "Juan"})
```

```
CALL {
  WITH u
  MATCH (u)-[:VIO {puntuacion: 5}]->(p:Pelicula)-[:DIRIGIDA_POR]->(d:Director)
  RETURN d AS director_favorito
}
```

```
WITH u, director_favorito
CALL {
  WITH u, director_favorito
  MATCH (director_favorito)-[:DIRIGIDA_POR]-(recomendacion:Pelicula)
  WHERE NOT (u)-[:VIO]->(recomendacion)
  RETURN recomendacion
}
```

```
RETURN recomendacion.titulo AS PeliculaRecomendada, director_favorito.nombre AS Director
```

4.4. Actualización de propiedades

4.4.1. Modificación de las propiedades de nodos y relaciones

```
EXAM=(2024J2.7)
```

La función SET se emplea dentro de un MATCH para modificar propiedades de un nodo.

El operador `+` o `||` se puede emplear dentro de la función `SET` para añadir un elemento a una lista.

```
MATCH (j:Juego {titulo: 'Alone in the Dark'})
SET j.etiquetas = j.etiquetas + 'Aventura'
RETURN j.etiquetas
```

4.4.2. Eliminación de las propiedades de nodos y relaciones

`REMOVE` se emplea dentro de un `MATCH` para eliminar propiedades de un nodo.

4.5

4.5.3. Borrado en cascada EXAM=(2025S0.11)

Para borrar todos los nodos y relaciones de una base de datos Neo4j se puede ejecutar la siguiente sentencia:

```
MATCH (n) DETACH DELETE n;
```

Esta sentencia funciona porque separa los nodos y después los borra. Si se intentara borrar sin separar, es decir, sin la cláusula `DETACH`, daría error.

Licencia y atribución

License CC BY 4.0

Este trabajo está licenciado bajo la licencia **Creative Commons Atribución 4.0 Internacional**.

© 2026, Colaboradores de apuntes-muicd-uned.